

Professor: Jackson Henrique

Alternante: William da Silva Matsunaga

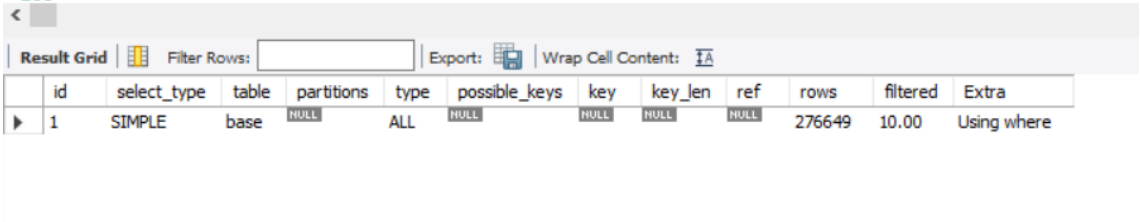
Relatório de desempenho usando o explain e indexação

Este relatório busca apresentar a implementação de indexação em um banco de dados e os resultados de melhoria de desempenho obtidos a partir da análise de consultas utilizando o comando explain.

No contexto deste trabalho, foi disponibilizada uma base de dados, contendo aproximadamente 291.000 registros. Essa tabela não possui índices definidos, o que faz com que o sistema de gerenciamento de banco de dados realize varreduras completas na tabela para executar as consultas, aumentando significativamente o tempo de resposta.

Parte 01 – Análise inicial (sem índices)

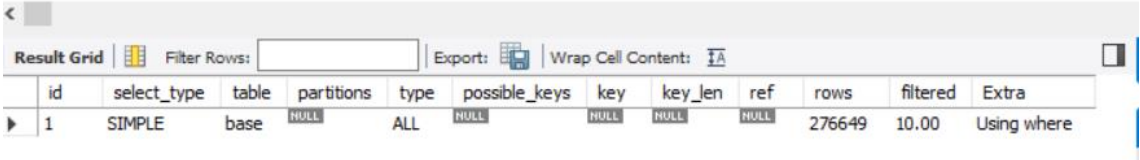
```
263 -- Exercícios
264
265 -- Parte 1 - Análise inicial (sem índices)
266
267 • explain select * from base where (ano = '2021');
268
```



	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	base	NULL	ALL	NULL	NULL	NULL	NULL	276649	10.00	Using where

Nesta primeira consulta, observa-se que o valor da coluna type está como “ALL”, o que indica que a consulta está realizando uma varredura completa na tabela, lendo todas as suas linhas. Isso pode ser observado na coluna rows, na qual o MySQL estima a leitura de 276649 linhas no total. Além disso, percebe-se que nenhum índice está sendo utilizado, pois as colunas possible_keys e key aparecem com valor NULL.

```
268
269 • explain select * from base where (SG_UF = 'RO');
270
271
272
```



	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	base	NULL	ALL	NULL	NULL	NULL	NULL	276649	10.00	Using where

Da mesma forma para esta consulta utilizando o explain:

- **type:** ALL
- **rows:** 276649
- **possible_keys:** null
- **key:** null

```
270
271 • explain select * from base where (evolucao = 'Óbito');
272
273
274
```

Result Grid												
Filter Rows:												
Export: Wrap Cell Content:												
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	base	NULL	ALL	NULL	NULL	NULL	NULL	276649	10.00	Using where

- **type:** ALL
- **rows:** 276649
- **possible_keys:** null
- **key:** null

Verificando tempo de consulta utilizando o “select * from base where (ano = '2021')”

```
✓ 179 21:44:10 select * from base where (ano = '2021') LIMIT 0, 1000 1000 row(s) returned 0.297 sec / 0.000 sec
```

- Segundos: 0.297 sec

Parte 2 – Criação de índices simples

Nesta fase, foi realizada criação de índices nas colunas ano, sg_uf e evolução da tabela base, conforme apresentado na imagem abaixo.

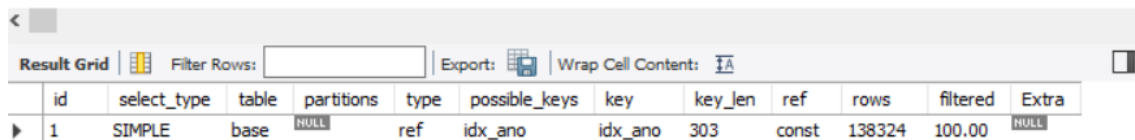
275	-- Parte 2 - Criação de índices simples
276	
277 •	create index idx_ano on base (ano);
278	
279 •	create index idx_sg_uf on base (sg_uf);
280	
281 •	create index idx_evolucao on base (evolucao);
282	
283	

Output			
Action Output			
#	Time	Action	Message
177	21:42:39	select * from base where (ano = '2021') LIMIT 0, 1000	Error Code: 1054. Unknown column 'ano' in 'where clause'
178	21:43:55	select * from base where (ano = '2021')	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL se...
179	21:44:10	select * from base where (ano = '2021') LIMIT 0, 1000	1000 row(s) returned
180	21:50:11	create index idx_ano on base (ano)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
181	21:50:16	create index idx_sg_uf on base (sg_uf)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
182	21:50:20	create index idx_evolucao on base (evolucao)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

Parte 3 – Análise após indexação

Após a realização da indexação, no índice `idx_ano` podemos observar que realmente foi aplicada tal código, como podemos observar abaixo na reexecução do comando `explain` na coluna `ano`.

```
264
265 -- Parte 1 - Análise inicial (sem índices)
266
267 • explain select * from base where (ano = '2021');
268
269 • explain select * from base where (SG_UF = 'RO');
270
271 • explain select * from base where (evolucao = 'Óbito');
272
273 • select * from base where (ano = '2021');
274
```

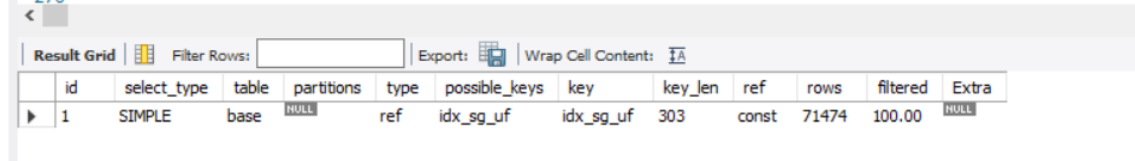


	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	base	NULL	ref	idx_ano	idx_ano	303	const	138324	100.00	NULL

Após a criação do índice e a reexecução do comando `EXPLAIN`, observa-se uma mudança no plano de execução da consulta. O valor da coluna `type` passou de “ALL” para “ref”, indicando que o MySQL passou a utilizar um índice não único para localizar os registros, evitando a varredura completa da tabela. Além disso, na coluna `possible_keys` aparece o índice `idx_ano`, indicando que ele pode ser utilizado na consulta, e na coluna `key` também é exibido `idx_ano`, confirmando que o índice está sendo efetivamente utilizado pelo MySQL. Outro ponto importante é a coluna `rows`, que agora apresenta uma estimativa de 138324 linhas a serem analisadas, número menor em comparação à consulta sem índice onde antes eram 276694 linhas, demonstrando uma melhoria no desempenho da busca.

Índice `idx_sg_uf`:

```
268
269 • explain select * from base where (SG_UF = 'RO');
270
```



	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	base	NULL	ref	idx_sg_uf	idx_sg_uf	303	const	71474	100.00	NULL

Antes / depois

Linhas consultadas: 276649 / 71474

Índice idx_evolucao

```
270
271 • explain select * from base where (evolucao = 'Óbito');
272
```

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	base	NULL	ref	idx_evolucao	idx_evolucao	303	const	138324	100.00	NULL

Antes / depois

Linhas consultadas: 276649/ 138324

select * from base where (ano = '2021');

```
272
273 • select * from base where (ano = '2021');
```

ANO	DT_SIN_PRI	MES_SIN_PRI	SEM_PRI	SG_UF_NOT	ID_MUNICIP	CS_SEXO	DT_NASC	NU_IDADE_N	FADIA_IDADE_N	CS_GESTANT	CS_RACA	CS_ESCOL_N	SG_UF	ID_MUN_RESI	C
2021	2021-01-03	Janeiro	1	PA	SANTAREM	Masculino	14/08/1946	74	70-79		Parda	Medio	PA	SANTAREM	Ur
2021	2021-01-07	Janeiro	1	AM	MANAUS	Feminino	17/12/2020	1	0-9		Parda		AM	MANAUS	Ur
2021	2021-01-07	Janeiro	1	TO	AUGUSTINOPOLIS	Masculino	15/12/1933	87	80-89		Branca		TO	SAMPAIO	Ur
2021	2021-01-04	Janeiro	1	AM	MANAUS	Feminino	18/05/1991	29	20-29	Nao	Parda	Medio	AM	MANAUS	Ur

base 1 x

Output

#	Time	Action	Message	Duration / Fetch
1	21:06:20	use bd_sars_20_21		0.016 sec
2	21:07:32	select * from base where (ano = '2021') LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.016 sec

Podemos verificar também após a execução do comando select, houve uma diminuição grotesca em relação ao tem para mostragem dos itens, passando de **0.297 para 0.000 segundos**.

Parte 4 – Índice composto

```
287 • create index idx_ano_sguf on base (ano, sg_uf);
288
289 • select * from base where (ANO = '2021') AND (SG_UF = 'RO');
290
291 • explain select * from base where (ano = '2021') and (SG_UF = 'RO');
292
293
```

ANO	DT_SIN_PRI	MES_SIN_PRI	SEM_PRI	SG_UF_NOT	ID_MUNICIP	CS_SEXO	DT_NASC	NU_IDADE_N	FADIA_IDADE_N	CS_GESTANT	CS_RACA	CS_ESCOL_N	SG_UF	ID_MUN_RESI	C
2021	2021-01-07	Janeiro	1	RO	JI-PARANA	Feminino	19/06/1979	41	40-49	Nao	Parda		RO	PORTO VELHO	Ur
2021	2021-01-03	Janeiro	1	RO	OURO PRETO DO OESTE	Feminino	30/08/1961	59	50-59	Nao	Parda		RO	OURO PRETO DO OESTE	Ru
2021	2021-01-12	Janeiro	2	RO	BURITIS	Masculino	13/01/1964	56	50-59		Parda		RO	BURITIS	Ur
2021	2021-01-16	Janeiro	2	RO	OURO PRETO DO OESTE	Feminino	06/02/1947	73	70-79	Nao	Branca		RO	OURO PRETO DO OESTE	Ur
2021	2021-01-29	Janeiro	4	RO	BURITIS	Feminino	23/06/1977	43	40-49		Parda	Fundamental 2	RO	BURITIS	Ur
2021	2021-02-02	Fevereiro	5	RO	PORTO VELHO	Masculino	01/01/1950	71	70-79		Branca	Fundamental 1	RO	OURO PRETO DO OESTE	Ur
2021	2021-02-03	Fevereiro	5	RO	PORTO VELHO	Feminino	04/12/1990	30	30-39	Sim	Parda	Medio	RO	PORTO VELHO	Ur

base 8 x

Output

#	Time	Action	Message	Duration / Fetch
6	21:36:46	SELECT * FROM base WHERE (ANO = '2021') AND (SG_UF = 'RO') LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.015 sec
7	21:39:29	select * from base LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
8	21:39:42	SELECT * FROM base WHERE (ANO = '2021') AND (SG_UF = 'RO') LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.016 sec
9	21:44:29	explain select * from base where (idx_ano_sguf)	Error Code: 1054. Unknown column 'idx_ano_sguf' in 'where clause'	0.000 sec
10	21:46:39	explain select * from base where (ano = '2021') and (SG_UF = 'RO')	1 row(s) returned	0.000 sec / 0.000 sec
11	21:48:13	select * from base where (ANO = '2021') AND (SG_UF = 'RO') LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec

A ordem das colunas no índice foi definida colocando ano primeiro e sg_uf em seguida, pois a consulta apresentada utiliza inicialmente a condição sobre ano e depois sobre sg_uf.

```

290
291 • explain select * from base where (ano = '2021') and (SG_UF = 'RO');
292

```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	base	NULL	ref	idx_ano,idx_sg_uf,idx_ano_sguf	idx_ano_sguf	606	const,const	58554	100.00	NULL

Observa-se que na coluna possible_keys aparecem os índices idx_ano, idx_sg_uf e idx_ano_sguf, indicando que todos poderiam ser utilizados na consulta.

Conclusão

A análise com o comando EXPLAIN demonstrou que a criação de índices melhora o desempenho das consultas. Inicialmente, o banco realizava varredura completa na tabela, analisando todas as linhas. Após a indexação, as consultas passaram a utilizar os índices, reduzindo a quantidade de registros analisados e tornando a execução mais eficiente.